

Informe

Análisis de Patrones y Arquetipos Colegio Bernardo Ohiggins

Docente: ISRAEL ALEJANDRO VILLAGRA RIQUELME

Integrantes:

- Cristobal Huinca Aravena

1. Introducción

Este documento describe los patrones de diseño, patrones arquitectónicos y arquetipos usados en la plataforma de libro de clases digital del Colegio Bernardo O'Higgins.

La solución se construyó con cuatro componentes principales:

- **frontend-libro clases:** aplicación React empaquetable como componente NPM.
- **bff-libroclases:** Backend For Frontend que orquesta datos.
- **ms-académico:** microservicio de gestión académica.
- **ms-asistencia:** microservicio de asistencias y anotaciones.

2. Patrones Arquitectonicos

Microservicios

La aplicación se separó en microservicios independientes para dividir responsabilidades:

- **ms-académico:**
 - Administra estudiantes, cursos, asignaturas, evaluaciones y notas.
- **ms-asistencia:**
 - Administra asistencias y anotaciones.

Cada microservicio tiene su propio proyecto, dependencias, base H2 local, controladores, servicios, repositorios y pruebas.

Ventajas:

- Separación clara de responsabilidades.
- Fácil mantenimiento por módulo.
- Cada componente puede versionarse y probarse de forma independiente.
- Facilita explicar la arquitectura en la defensa oral.

Backend For Frontend

El BFF se implementó en **bff-libroclases**. Su objetivo es evitar que el frontend llame directamente a todos los microservicios.

Funciones principales:

- Recibir solicitudes del frontend.
- Consultar **ms-académico** y **ms-asistencia**.
- Construir respuestas con más detalle, por ejemplo perfil de estudiante, anotaciones detalladas y asistencias detalladas.
- Completar automáticamente el **curso** de un estudiante al registrar anotaciones o asistencias.

Beneficio:

El frontend queda más simple porque consume una API pensada para sus pantallas.

3. Patrones De Diseño Backend

Repository Pattern

Se usa mediante interfaces que extienden **JpaRepository**.

Ejemplos:

- **CursoRepository**
- **EstudianteRepository**
- **AsignaturaRepository**
- **AsistenciaRepository**
- **AnotacionRepository**

Responsabilidad:

- Separar la lógica de persistencia de la lógica de negocio.
- Evitar consultas SQL manuales para operaciones básicas.
- Permitir testear servicios con repositorios simulados mediante Mockito.

Service Layer Pattern

Cada microservicio tiene clases `@Service` encargadas de la lógica de negocio.

Ejemplos:

- **EstudianteService**
- **CursoService**
- **EvaluacionService**
- **AsistenciaService**
- **AnotacionService**
- **LibroClasesService**

Responsabilidad:

- Validar reglas de negocio.
- Buscar entidades relacionadas.
- Evitar que los controladores contengan lógica compleja.
- Centralizar operaciones importantes.

DTO Pattern

Se usan DTOs para separar los datos que entran y salen por la API de las entidades JPA.

Ejemplos:

- **EstudianteRequest**
- **CursoRequest**
- **AnotacionCreateRequest**
- **PerfilEstudianteDto**
- **AsistenciaDetalleDto**

Ventajas:

- Evita exponer directamente las entidades.
- Permite validar campos con `@Valid`.
- Hace más clara la documentación Swagger.
- Reduce el acoplamiento entre capas.

MVC / REST Controller

La aplicación usa una división por capas:

- **Controller**: recibe peticiones HTTP.
- **Service**: procesa reglas de negocio.
- **Repository**: accede a datos.
- **Entity/DTO**: representa datos.

Esto mantiene el código ordenado y facilita las pruebas.

Dependency Injection

Spring inyecta dependencias con `@Autowired`.

Ejemplo:

- Un controller usa un service.
- Un service usa un repository.
- Un client del BFF usa un **RestClient**.

Esto evita crear objetos manualmente y permite reemplazarlos por mocks en pruebas unitarias.

Centralized Exception Handling

Se usa `@RestControllerAdvice` en `GlobalExceptionHandler`.

Responsabilidad:

- Capturar errores de negocio.
- Devolver respuestas HTTP consistentes.
- Evitar repetir manejo de errores en cada controller.

Circuit Breaker

En el BFF se usa Resilience4j con `@CircuitBreaker`.

Responsabilidad:

- Proteger al BFF cuando un microservicio no responde.
- Activar fallbacks para consultas.
- Evitar que una falla de un microservicio detenga completamente la experiencia del usuario.

Ejemplo de explicación:

Si **ms-académico** está apagado, el BFF puede devolver datos parciales o valores por defecto en vez de fallar completamente en endpoints de consulta.

4. Patrones Frontend

Component-Based Architecture

React divide la interfaz en componentes reutilizables.

Ejemplos:

- **Navbar**
- **Badge**
- **InfoCard**
- **RoleSelector**
- **RoleGuard**

Page Component Pattern

Cada pantalla principal tiene su propio componente de página.

Ejemplos:

- **Home**
- **PerfilEstudiante**
- **AnotacionDetalle**
- **AsistenciaDetalle**
- **RegistrarAnotacion**
- **RegistrarAsistencia**

Context API

Se usa **RoleContext** para guardar el rol actual del usuario.

Permite que varias partes del frontend conozcan el rol sin pasar props manualmente.

RoleGuard

RoleGuard controla qué opciones se muestran según el rol seleccionado.

Roles usados:

- Profesor.
- Inspector.
- Apoderado.
- Estudiante.

API Client Pattern

El archivo **bffApi.js** concentra las llamadas HTTP hacia el BFF.

Ventajas:

- Las páginas no construyen URLs manualmente.
- Si cambia el endpoint del BFF, se actualiza en un solo lugar.
- Facilita pruebas unitarias con mocks.

5. Arquetipo Maven

Se generó un arquetipo Maven a partir de **ms-académico**.

Objetivo:

- Crear nuevos microservicios con una estructura base similar.
- Reutilizar convenciones de paquetes, dependencias, configuración y capas.

Comando usado:

```
.\mvnw.cmd -s ".\archetype-settings.xml" archetype:create-from-project
```

Resultado:

- Arquetipo generado en **target/generated-sources/archetype**.
- Instalado localmente como **cl.bohiggins:ms-académico-archetype:0.0.1-SNAPSHOT**.
- Verificado creando el proyecto demo **ms-prueba-arquetipo**.

6. Empaquetado NPM

El frontend se preparó como paquete NPM reutilizable.

Elementos relevantes:

- **package.json** con **private: false**.
- **src/lib/index.js** como punto de entrada.
- **vite.lib.config.js** para build de librería.
- Script **npm run build:lib**.
- Validación con **npm pack**.

Esto permite reutilizar componentes React en otros proyectos.

7. Conclusiones

En conclusion, los patrones y arquetipos utilizados permitieron construir una solucion mas ordenada, mantenible y facil de explicar. La separacion entre frontend, BFF y microservicios ayuda a dividir responsabilidades, mientras que patrones como Repository, Service Layer y DTO organizan mejor el backend y separan la logica de negocio, la persistencia y los datos expuestos por la API.

El BFF aporta una capa intermedia que simplifica el trabajo del frontend, ya que une informacion de los microservicios y entrega respuestas mas completas para las pantallas. Ademias, el uso de Circuit Breaker mejora la tolerancia a fallos ante posibles caidas de servicios.

Por otro lado, el arquetipo Maven y el empaquetado NPM demuestran reutilizacion tecnica: el primero permite generar nuevos microservicios con una estructura base, y el segundo permite reutilizar componentes del frontend en otros proyectos. Con esto, la solucion cumple los criterios de la rubrica y queda preparada para futuras mejoras.

La solución aplica patrones de diseño y arquitectura alineados con la rúbrica:

- Microservicios.
- BFF.
- Repository.
- Service Layer.
- DTO.
- MVC REST.
- Circuit Breaker.
- Context API.
- RoleGuard.
- Arquetipo Maven.
- Empaquetado NPM.